Fibonacci Series –

Space Complexity (Detailed Analysis)

Program:

```
// Fibonacci Series through Recursion

// This code prints the Fibonacci series using recursion.
The Fibonacci series is a series of numbers where the next
number is the sum of the previous two numbers. The first two
numbers of the series are 0 and 1. The series is as follows:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, etc.

#include <iostream>

using namespace std;

int fibonacci(int n) // Function to calculate the nth
Fibonacci number
{
    if (n == 0 || n == 1) // Base case
    {
        return n; // If n is 0 or 1, return n
    }
    return fibonacci(n - 1) + fibonacci(n - 2); // Recursive
call : nth Fibonacci number is the sum of (n-1)th and (n-
2)th Fibonacci number
}

int main() // Main function
{
    int n;                                  // Number of
terms
    cout << "Enter the number of terms: "; // Input the
number of terms
    cin >> n;                               // Store the
number of terms
```

```
    cout << "The Fibonacci series is: ";   // Output the
Fibonacci series
    for (int i = 0; i < n; i++)           // Loop to print
the first n Fibonacci numbers
    {
        cout << fibonacci(i) << " "; // Print the ith
Fibonacci number
    }
    return 0; // Return statement
}
```

Space Complexity :

1. ***Input Space***

Input space refers to the memory allocated for the variables and parameters provided to run the algorithm, independent of the algorithm's internal execution steps.

- ***Holistic Input Space***: *In* $main$ *function, allocated memory for an integer* n *(the user input) and a loop counter* i.
  Because these are simple primitive types (usually 4 bytes each in $C++$*), they take a constant amount of memory regardless of how large the input number grows. Therefore, the holistic input space is* $\boldsymbol{O}(\mathbf{1})$.

- ***Parametric Space***: *In the* $fibonacci(int\ n)$ *function, the parameter* n *is passed by value. Every time the function is called, memory is allocated to hold this integer within the activation record (stack frame) of that specific call.*
  Because it is a single integer, the parametric space per call is $\boldsymbol{O}(\mathbf{1})$.

3. Auxiliary Space (VERY IMPORTANT PART)

Now the real complexity comes from recursion.

Understanding recursion stack

Each call to:

```
    int n;
    cout << "Enter the number of terms: ";
    cin >> n;
    cout << "The Fibonacci series is: ";
    for (int i = 0; i < n; i++)
    {
        cout << fibonacci(i) << " ";
    }
```

$creates\ a\ stack\ frame\ containing$:

- $parameter\ n$
- $return\ address$
- $temporary\ storage$

$Each\ frame\ =\ \Theta(1) space\ per\ recursive\ call$

Maximum recursion depth

$Even\ though\ Fibonacci\ creates\ \boldsymbol{two\ recursive\ calls}, they\ are$

$\boldsymbol{not\ simultaneous\ in\ stack}.$

$Execution\ is\ depth - first:$

$\mathbf{1})\ \boldsymbol{When\ fibonacci(0)\ , we\ get\ a\ single\ node\ i.e.}:$

$$\boldsymbol{fib(0)}$$

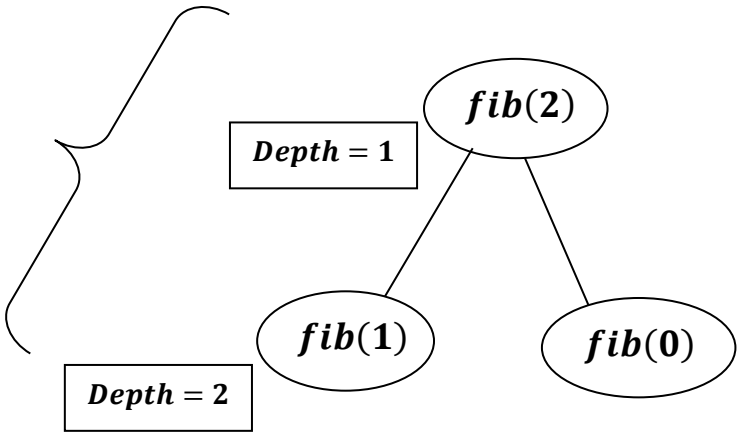
$\boldsymbol{Depth\ of\ the\ node\ is\ 1, hence\ space\ complexity\ is\ O(1)}.$

$\mathbf{2})\ \boldsymbol{When\ fibonacci(1)\ , we\ get\ a\ single\ node\ i.e.}:$

$$\boldsymbol{fib(1)}$$

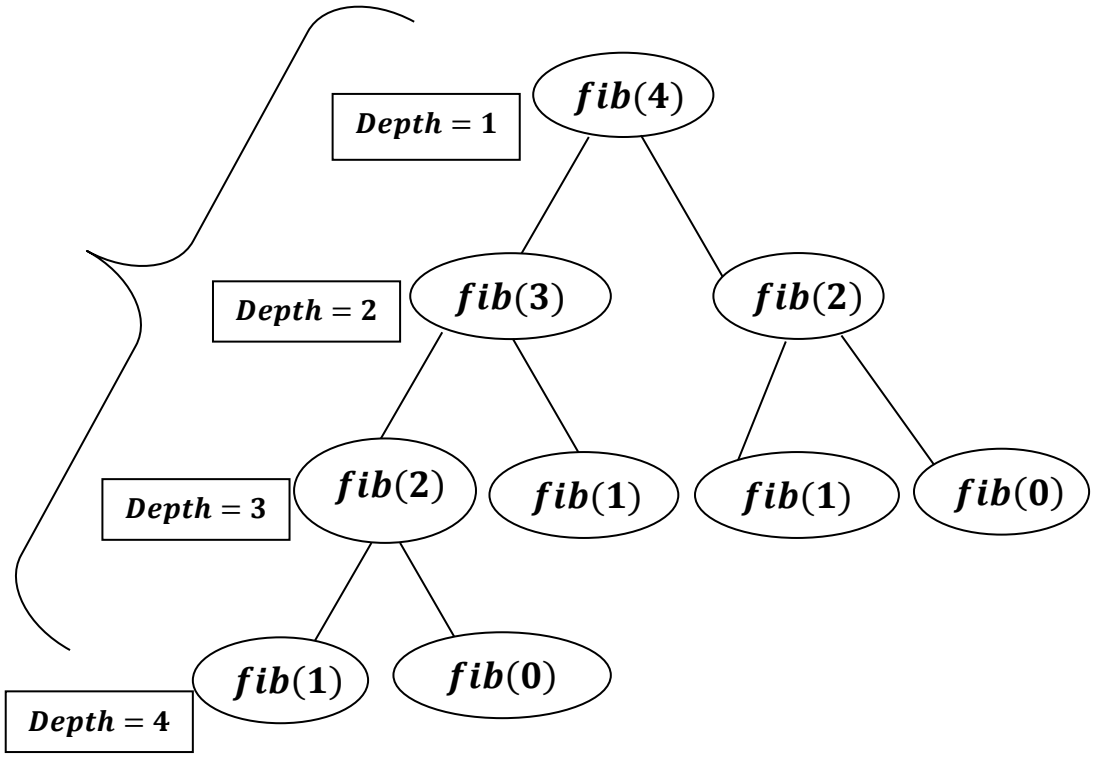
$\boldsymbol{Depth\ of\ the\ node\ is\ 1, hence\ space\ complexity\ is\ O(1)}.$

$3)$ $When\ fibonacci(2)\ ,we\ get\ \ recursion\ tree$:



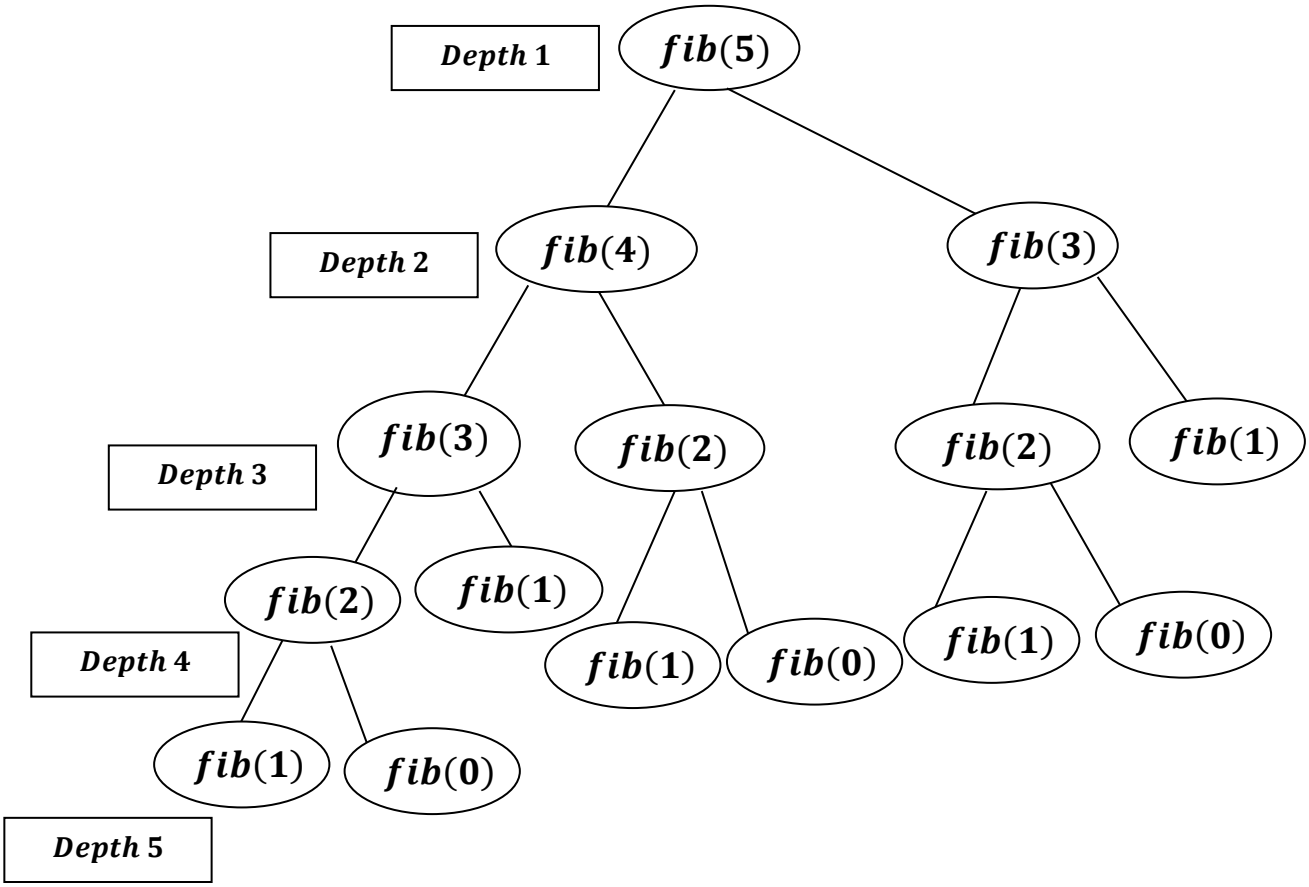
$Maximum\ Depth\ of\ the\ tree\ is\ 2, hence\ space\ complexity\ is\ O(2)$.

$4)$ $When\ fibonacci(4)\ ,we\ get\ recursion\ tree$:



$\boldsymbol{Maximum\ Depth\ of\ the\ tree\ is\ 4, hence\ space\ complexity\ is\ O(4)}$.

$\boldsymbol{In\ Addition: fibonacci(5)\ , we\ get\ recursion\ tree \rightarrow}$



$\boldsymbol{Maximum\ Depth\ of\ the\ tree\ is\ 5, hence\ space\ complexity\ is\ O(5)}$.

$\boldsymbol{Hence\ if\ it\ starts\ from\ n, the\ \ maximum\ depth(height) of\ the\ tree\ will\ be\ n.}$

$$\boldsymbol{Hence\ , we\ take\ here\ the\ Maximum\ Depth = n}$$

$$\therefore \boldsymbol{Total\ Auxiliary\ Space = Maximum\ Depth \times Space\ per\ Recursive\ Call.}$$

$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$
$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$\therefore$ $\boldsymbol{Total\ Space\ Complexity}$:

$$\boldsymbol{Input\ Space + Auxiliary\ Space = \ \Theta(1) + \Theta(n)}$$
$$= \ \boldsymbol{\Theta(n)}.$$